Numerical open source software packages for ODEs & PDEs*

Duc-Lam Duong† and Anotida Madzvamuse

Lecture notes for the UK-APASI training program, ICMS, Edinburgh
Spring 2021

Abstract

This file contains online (mostly open) sources of some software packages and solvers for the numerical solution of ordinary differential equations (ODEs) and partial differential equations (PDEs)[1].

Contents

1 MATLAB **2**
1.1 Setup and general information 2
1.2 ODE solvers 2
1.3 PDE solvers 3

2 Octave and Scilab **4**
2.1 Setup and general information 4
2.2 ODE solvers 4
2.2.1 Octave 4
2.2.2 Scilab 5
2.3 PDE solvers 5
2.3.1 Octave 5
2.3.2 Scilab 5

3 Python, Spyder and Anaconda **6**
3.1 Setup and general information 6
3.2 ODE solvers 6
3.3 PDE packages and solvers 7

*DISCLAIMER: This collection is for educational purposes. The authors do not claim authorship nor expertise in any of the software packages listed!

†Please send feedback or updates to `dlam.duong@gmail.com`

[1]This file was used as a supplementary document for participants of the ICMS UK-APASI program `https://www.icms.org.uk/events/workshops/UKAPASI`. Support from the UKRI-GCRF-EPSRC grant (EP/T00410X/1) is acknowledged.

4 R **7**
4.1 Setup and general information 7
4.2 ODE packages and solvers 8
4.3 PDE solvers 8

5 Julia **8**
5.1 Setup 8
5.2 ODE packages and solvers 9
5.3 PDE packages and solvers 9

6 C++ **9**
6.1 Setup 9
6.2 ODE solvers 9
6.3 PDE solvers 10

7 Other open sources **10**

1 MATLAB

1.1 Setup and general information

MATLAB is NOT an open source software. However we list it here due to its popularity and convenience. It is also a based language for some others listed later on.

MATLAB is a product of the mathematical computing software company MathWorks. MATLAB can be downloaded and installed from here: `https://www.mathworks.com/downloads/`[2]. You will need to sign up to the MathWorks website to be able to download and use their products. Unfortunately, MATLAB is not free and you need to have a license, either by yourself or through your institution[3].

With a licensed account, you can also use MATLAB online directly on the website `https://matlab.mathworks.com/` without having to install it on your computer.

1.2 ODE solvers

There are quite a few MATLAB solvers for ODEs with different accuracies and efficiencies. Below are some of the most common ones. We refer to the corresponding MathWorks pages for detailed syntaxes and examples of each solver.

[2]Note that the site domain might change slightly depending on your region, for example `https://uk.mathworks.com/` if you are in the UK.

[3]You can however start a 30-day free trial here `https://www.mathworks.com/campaigns/products/trials.html?s_tid=hp_trial_download`. If you are a student, you may also get a much cheaper discounted price.

- `ode45`, `https://uk.mathworks.com/help/matlab/ref/ode45.html`. This solver performs well with the majority of ODE problems and generally should be the first one to try. It may however not be efficient with stiff ODEs.

- `ode23`, `https://uk.mathworks.com/help/matlab/ref/ode23.html`. This solver may have better performance compared to `ode23` for ODEs with moderate stiffness.

- `ode15s`, `https://uk.mathworks.com/help/matlab/ref/ode15s.html`. This solver is the first choice for most stiff ODEs. It can also be used to solve differential algebraic equations (DAEs).

A full list containing other solvers such as `ode113, ode23s, ode15i`,... and their comparison can be found at the page `https://uk.mathworks.com/help/matlab/math/choose-an-ode-solver.html`. For more details and recommendations on when to use each solver, see *Shampine and Reichelt, The MATLAB ODE Suite, SIAM Journal on Scientific Computing* [7].

1.3 PDE solvers

For 1D parabolic and elliptic PDEs, we can use MATLAB solver `pdepe`. See here `https://uk.mathworks.com/help/matlab/partial-differential-equations.html`.

For high-dimensional PDEs, we use the PDE toolbox. This toolbox in MATLAB provides solvers for many PDEs[4] (including structural mechanics, heat transfer and electromagnetics) using finite element method. The general information can be found here `https://uk.mathworks.com/help/pde/` and here `https://uk.mathworks.com/help/pde/pde-problem-setup.html`. To see which equations you can solve using the PDE toolbox, see `https://uk.mathworks.com/help/pde/ug/equations-you-can-solve.html`. There you can also find the information on how to put your problem in the correct form to be able to use the toolbox and its objects.

The `PDEModel` object (whose information can be found here `https://uk.mathworks.com/help/pde/ug/solve-problems-using-pdemodel-objects.html`) has two solvers that can be used for the majority of PDEs.

- `solvepde`, for all problems except for eigenvalue problems. `https://uk.mathworks.com/help/pde/ug/pde.pdemodel.solvepde.html`.

- `solvepdeeig`, for eigenvalue problems. `https://uk.mathworks.com/help/pde/ug/pde.pdemodel.solvepdeeig.html`.

These solvers return solutions for the PDEs as objects:

[4] We note that, by using for example the method of lines (FDM, FEM, FVM), many PDEs can be transformed to a system of ODEs, thus ODE solvers can be used to solve PDEs!

- `StationaryResults`: time-independent PDE solution and derived quantities, `https://uk.mathworks.com/help/pde/ug/pde.stationaryresults.html`.
- `TimeDependentResults`: time-dependent PDE solution and derived quantities, `https://uk.mathworks.com/help/pde/ug/pde.timedependentresults.html`.
- `EigenResults`: PDE eigenvalue solution and derived quantities, `https://uk.mathworks.com/help/pde/ug/pde.eigenresults.html`.

To evaluate the solution at one point, use `pdeval`. To help with visualizing the mesh and solution, use plot functions such as `pdeviz, pdeplot, pdegplot, pdeplot3D`. The reader can look up their syntaxes and examples on the MATLAB documentation webpage `https://uk.mathworks.com/help/index.html`.

Finally, to find the full documentation for the PDE toolbox, check this out: `https://uk.mathworks.com/help/pdf_doc/pde/index.html`. See also the nice book by Linge and Langtangen (2016) – a gentle introduction to computer programming in MATLAB/Octave – [3] which is available freely at `https://www.springer.com/gp/book/9783319324517` (for an online tutorial of the book, see `https://hplgit.github.io/prog4comp/`). The authors also make the code available in Python [4].

2 Octave and Scilab

2.1 Setup and general information

GNU Octave and Scilab are created as open source alternatives for MATLAB. They are completely free and most of the codes in MATLAB can be run on Octave or Scilab.

Download Octave here `https://www.gnu.org/software/octave/download`.

Download Scilab here `https://www.scilab.org/download/`.

As with MATLAB, you can run Octave and Scilab codes online without having to download the software to your computer. Check out here `https://octave-online.net/` (for Octave) and here `https://cloud.scilab.in/` (for Scilab). See also the online manual here `https://octave.org/doc/v6.2.0/` (for Octave) and `https://wiki.scilab.org/Documentation` (for Scilab).

2.2 ODE solvers

2.2.1 Octave

Octave has its own standard ODE solver, called `lsole`. Check out `https://octave.org/doc/v6.1.0/Ordinary-Differential-Equations.html` for the syntax and examples.

A good thing about Octave is that it can run the majority of the MATLAB solvers, including `ode45, ode23, ode15s` mentioned above. See a list of MATLAB-compatible solvers at `https://octave.org/doc/v6.1.0/Matlab_002dcompatible-solvers.html`.

2.2.2 Scilab

The standard ODE built-in solver in Scilab is `ode`. See at `https://help.scilab.org/docs/6.1.0/en_US/ode.html`. Other solvers (including the ones for DAEs) can be found in the following Scilab Help documentation `https://help.scilab.org/docs/6.1.0/en_US/index.html`.

The book by Wouwer, Saucez and Vilas (2014) [10] contains implementations of various ODEs and PDEs in MATLAB, Octave and Scilab. See also Linge and Langtangen (2016) [3].

2.3 PDE solvers

2.3.1 Octave

The most popular Octave PDE packages[5] is called `fem-fenics` which solves PDEs using finite element method, based on fenics. See here for a concise tutorial and examples `https://wiki.octave.org/Fem-fenics` and here for download and updates `https://octave.sourceforge.io/fem-fenics/index.html`.

The other package, `bim`, see `https://wiki.octave.org/Bim_package`, solves PDEs based on finite volume method. See here for download and updates `https://octave.sourceforge.io/bim/index.html`.

2.3.2 Scilab

The Scilab PDE toolbox `scipde` can be found at `http://forge.scilab.org/index.php/p/scipde/`. This toolbox provides a couple of functions to solve PDEs, `scipde_heat1Dsolve` (solve 1D diffusion equations) and `scipde_heat1Dsteady` (stationary state of 1D diffusion equations). See also `https://help.scilab.org/docs/6.1.0/en_US/PDE.html`.

For using finite element method and finite volume method in Scilab, we refer to `https://wiki.scilab.org/Finite%20Elements%20in%20Scilab` and `https://wiki.scilab.org/Finite%20Volumes%20in%20Scilab`.

See also the books that have been mentioned: [3, 10].

[5] A list of packages for different purposes can be found at `https://octave.sourceforge.io/packages.php`.

3 Python, Spyder and Anaconda

3.1 Setup and general information

Python is an open-source and general-purpose programming language and interpreter (home page: `https://www.python.org/`).

For scientific computing and computational modelling, as with other languages, we need additional packages that are not part of the Python standard library. `NumPy` and `SciPy` are two main packages for scientific computing with Python. Installation information of these packages can be found here `https://numpy.org/install/` and here `https://www.scipy.org/install.html`.

Spyder is a free and powerful interactive development environment for Python.

Download Spyder here `https://www.spyder-ide.org/`,

Installation instruction can be found at `https://docs.spyder-ide.org/current/installation.html`.

Anaconda is one of several Python distributions and is recommended for scientific computing. Anaconda provides a large collection of libraries, including thousands of Python/R packages (including `NumPy` and `SciPy`). Spyder environment also comes as part of Anaconda. In short, *installing Anaconda will provide pretty much everything you need for scientific computing*!

Download Anaconda at `https://www.anaconda.com/products/individual`.

Installation instruction at `https://docs.anaconda.com/anaconda/install/`.

For more information regarding the installation of Python and its packages, Anaconda and Spyder, we refer to the following Github blog post

`http://bit.ly/python-anaconda-spyder`.

3.2 ODE solvers

There are quite a few ODE solvers in SciPy. The previously developed solvers include `odeint, ode` and `complex_ode`.

The new interface solvers include

- `OdeSolver`. Based class for ODE solvers.
- `solve_ivp`. Standard solver for an initial value problem.
- `RK23, RK45, DOP853, Radau`. Solvers based on Runge-Kutta methods of various orders.
- `BDF`. Implicit ODE solver based on backward-differentiation formulas.
- `LSODA`. Adams/BDF method with automatic stiffness detection and switching.
- `solve_bvp`. Solver for a boundary value problem.

All syntaxes for these solvers can be found here https://docs.scipy.org/doc/scipy/reference/integrate.html.

3.3 PDE packages and solvers

There is a rich list of PDE packages in Python. Below is a sample of them.

- FEniCS, a platform that enables users to quickly translate scientific models into efficient finite element code, which is also available in C++. See here https://fenicsproject.org/.
- FEMPy a pure-python package for solving PDEs based on finite element method. Visit https://fempy.readthedocs.io/en/latest/ for installation, updates and documentation. See also http://fempy.org/.
- FiPy a Python PDE solver based on finite volume method. See https://pypi.org/project/FiPy/.
- PyMFEM, a Python wrapper for MFEM, see https://github.com/mfem/PyMFEM.
- Py-pde a new Python package to solve PDEs, with emphasis on problems arising in the field of dynamical systems. See https://pypi.org/project/py-pde/.
- PySE a finite difference method toolbox. See http://pyfdm.sf.net.
- SyFi builds on top of the symbolic math library GiNaC and is based on finite element method, see https://pypi.org/project/SyFi/.

For an open-source book, we refer to the one by Linge and Langtangen [4] which is available for free at https://www.springer.com/gp/book/9783319812823 (see https://hplgit.github.io/prog4comp/ for an online tutorial of the book).

4 R

4.1 Setup and general information

R is a language and environment for statistical computing and graphics. R can be downloaded and installed for free at https://cran.r-project.org/bin/.

For more information on using R, we refer to the book by Hastie, Tibshirani and Friedman [2]. This book is a valuable resource for data analytics in R as it approaches techniques through their statistical underpinnings, describes examples of their use using R packages, and in many cases provides links to R packages that are specific to the problems discussed. The book is available online free of charge here https://web.stanford.edu/~hastie/ElemStatLearn/. See also the book [1] (again available online, at https://www.statlearning.com/) on Statistical Learning with applications in R.

4.2 ODE packages and solvers

- `deSolve`. `http://desolve.r-forge.r-project.org/`. This package contains several solvers for solving ODEs, DAEs, DDEs and even PDEs. It can deal with stiff and nonstiff problems. This includes some solvers that are common in other languages such as `ode, lsode, lsoda`. For a list of solvers together with their syntaxes and examples, see `https://www.rdocumentation.org/packages/deSolve/versions/1.28`. See also `https://cran.r-project.org/web/packages/deSolve/vignettes/deSolve.pdf`.
- `pracma`. `https://cran.r-project.org/package=pracma`. This package implements several adaptive Runge-Kutta solvers such as `ode23, ode23s, ode45`, or the Burlisch-Stoer algorithm to obtain numerical solutions to ODEs with higher accuracy. See the solver list here `https://www.rdocumentation.org/packages/pracma/versions/1.9.9`.

Several other packages for ODEs, Stochastic DEs, DAEs and so on can be found on the following page `https://cran.r-project.org/web/views/DifferentialEquations.html`.

4.3 PDE solvers

- `ReacTran`. This package provides functions for converting the PDEs into a set of ODEs and can discretise PDEs on cartesian, polar, cylindrical and spherical grids. Its main target is in the field of "reactive transport" modelling, but it can be used to solve PDEs of the three main types. See `https://CRAN.R-project.org/package=ReacTran`.
- `deSolve`. The package contains dedicated solvers for 1-D, 2-D and 3-D time-varying ODE problems as generated from PDEs (e.g. by `ReacTran`).
- `rootSolve`. The package contains optimized solvers for 1-D, 2-D and 3-D algebraic problems generated from (time-invariant) PDEs.

Note that, to date, one can only solve PDEs in R using finite difference method. We refer to the book [8] for an introduction to solving (partial) differential equations in R.

5 Julia

5.1 Setup

Julia is a free programming language that is well-suited for numerical analysis and computational science.

Download Julia at `https://julialang.org/downloads/`.

Documentation `https://docs.julialang.org/en/v1/`.

5.2 ODE packages and solvers

The package `DifferentialEquation.jl` (see `https://diffeq.sciml.ai/stable/`) covers most of what you need to solve a differential equation. This contains a rich list of solvers in DEs which include ODEs, Stochastic DEs, DAEs and PDEs.

- Tutorial `https://diffeq.sciml.ai/stable/tutorials/ode_example/`.
- `DifferentialEquation.jl` overview `https://diffeq.sciml.ai/stable/basics/overview/`.
- Algorithms and a list of ODE solvers `https://diffeq.sciml.ai/stable/solvers/ode_solve/` (including translation solvers from MATLAB/Python/R such as `ode45, ode23, ode15s` etc).

5.3 PDE packages and solvers

Julia is very well-built for solving PDEs. There are several PDE packages and solvers in Julia. The following is just a sample of the most common ones.

- `FEniCS.jl, JuliaFEM, JuAFEM.jl` solve PDEs using finite element method.
- `DiffEqOperators.jl` is part of the JuliaDiffEq ecosystem, to solve PDEs using finite difference method.
- `FiniteVolume.jl, VoronoiFVM.jl, JFVM.jl` solve PDEs using finite volume method.
- `FFTW.jl` and `ApproxFun.jl` solve PDEs using spectral method.

We refer to `https://github.com/JuliaPDE/SurveyofPDEPackages` for a review of PDE packages in Julia. Other useful information can be found here `https://nextjournal.com/sosiris-de/pde-2018` and at the official page for the package DifferentialEquation.jl, `https://diffeq.sciml.ai/stable/`.

6 C++

6.1 Setup

C++ is a free multipurpose programming language and an extension of the programming language C. For installation and documentation, we refer to `https://isocpp.org/`. Useful information can also be found here `http://www.cplusplus.com/`.

6.2 ODE solvers

The standard ODE library in C++ is `odeint`. For documentation and examples, we refer to `http://headmyshoulder.github.io/odeint-v2/`. See also `https://www.gnu.org/software/gsl/doc/html/` for many solvers in GNU Scientific Library that are also compatible with C++.

6.3 PDE solvers

There are several packages/solvers written in C++ for solving PDEs. Below is just a sample of them.

- `deal.II` offers a large collection of different finite elements of any order, with extensive documentation and tutorial programs. Visit `https://dealii.org/` for download, documentation and examples.
- `MFEM` features arbitrary high-order finite element meshes and spaces, support for a wide variety of discretizations, and emphasis on usability, generality, and high-performance computing efficiency. Check it out here `https://mfem.org/`.
- `FreeFEM++` is a popular 2D and 3D PDE solver. Visit the official webpage `https://freefem.org/` for download, documentation and examples. You can also run your code online here `https://freefem.org/tryit`. See also `https://github.com/FreeFem`.
- `GetFEM++` provides a framework for solving coupled systems of linear and nonlinear PDEs based on finite element method, see `http://getfem.org/`.
- `DUNE` is a free modular toolbox for solving PDEs using grid-based methods. The official DUNE project is at `dune-project.org`.
- `FEniCS` a platform that solves PDEs using finite elements, with Python and C++ interfaces.
- `jive` a quite new C++ toolbox to solve PDEs, based mainly on finite element method, see `https://software.dynaflow.com/jive/`

For reference, we refer to the enormous book Numerical Recipes in C [5]. The previous version of the book is available freely here `http://www.nrbook.com/a/bookcpdf.html`. For a more PDE-oriented book, we refer to Shapira [9].

For a list of FEM software packages, we refer to `https://en.wikipedia.org/wiki/List_of_finite_element_software_packages`.

7 Other open sources

There are many other software and open-source libraries that can be used to solve numerically ODEs and PDEs: FORTRAN, Mathematica, Maple, Java and so on. We list some packages below. Note that by the nature of the information, as with any other software in this document, it may be outdated over time. The reader is encouraged to look for more information/updates on the Internet.

- ODE and PDE solvers in FORTRAN https://people.sc.fsu.edu/~jburkardt/f77_src/odepack/odepack.html and https://www.osti.gov/biblio/6228751-pdeone-solutions-partial-differential-equations-cdc7600-fortran-iv.
- Hairer's Solvers in Fortran and MATLAB http://www.unige.ch/~hairer/software.html.
- PDE solver in Maple https://www.maplesoft.com/support/help/maple/view.aspx?path=pdsolve.
- ODE tool in Vose (a quantitative risk analysis and management software) https://www.vosesoftware.com/riskwiki/OrdinaryDifferentialEquationstool.php
- ODE solvers in Maxima https://maxima.sourceforge.io/docs/tutorial/en/gaertner-tutorial-revision/Pages/ODE0001.htm
- ALBERTA, an adaptive hierarchical finite element toolbox http://www.alberta-fem.de/

Other useful information and comparison between some software packages can be found in an article by Christopher Rackauckas [6], which can also be accessed online from the following blog post of the author
http://bit.ly/comparison-ode-solvers.

Appendix

This appendix contains some MATLAB code from a series of lectures by Prof. Fred Vermolen given at the UK-APASI Program, Workshop 4 (Modelling, Analysis, Numerical Methods and Applications of ODEs and SDEs) on 17/03/2021.

- Coronavirus https://uk.mathworks.com/matlabcentral/fileexchange/88922-simple-coronavirus-models.
- Game of life https://uk.mathworks.com/matlabcentral/fileexchange/88927-game-of-life.

References

[1] James G, Witten D, Hastie T, Tibshirani R. *An Introduction to Statistical Learning: With Applications in R*, Springer, 2013. Available online at https://www.statlearning.com/.

[2] Trevor Hastie, Robert Tibshirani and Jerome Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd edn, Springer, 2009. Available at https://web.stanford.edu/~hastie/ElemStatLearn/.

[3] Svein Linge and Hans Petter Langtangen, *Programming for computations–MATLAB/Octave*. Springer Nature, 2016. Available freely at `https://www.springer.com/gp/book/9783319324517`.

[4] Svein Linge and Hans Petter Langtangen, *Programming for computations–Python*. Springer Nature, 2016. Available freely at `https://www.springer.com/gp/book/9783319812823`.

[5] William H. Press, Saul Teukolsky, Brian P. Flannery, William T. Vetterling, *Numerical Recipes in C: The Art of Scientific Computing*, 3rd ed., 2007. The second edition is available at `http://www.nrbook.com/a/bookcpdf.html`.

[6] Christopher Rackauckas, *A Comparison Between Differential Equation Solver Suites In MATLAB, R, Julia, Python, C, Mathematica, Maple, and Fortran*, The Winnower 6:e153459.98975 (2018).

[7] Shampine, L. F. and Reichelt, M. W., *The MATLAB ODE Suite*, SIAM Journal on Scientific Computing, Vol. 18, 1997, pp. 1–22.

[8] Karline Soetaert, Jeff Cash, and Francesca Mazzia. *Solving Differential Equations in R*. Springer Science & Business Media, 2012.

[9] Yair Shapira. *Solving PDEs in C++: Numerical methods in a unified Object-oriented Approach*. Vol. 9. SIAM, 2012.

[10] Wouwer, A. V., Saucez, P., Vilas, C., *Simulation of ODE/PDE Models with MATLAB, OCTAVE and SCILAB: Scientific and Engineering Applications*. Springer, 2014.